# Gaslite

## Layer Zero Stargate V2 Gas Optimization Audit Report

## April 8, 2024

## Scope:

- https://github.com/stargate-protocol/stargate-v2/tree/main/packages/stg-evm-v2/src/interfaces
- https://github.com/stargate-protocol/stargate-v2/tree/main/packages/stg-evm-v2/src/libs
- https://github.com/stargate-protocol/stargate-v2/tree/main/packages/stg-evm-v2/src/messaging
- https://github.com/stargate-protocol/stargate-v2/tree/main/packages/stg-evm-v2/src/usdc
- https://github.com/stargate-protocol/stargate-v2/tree/main/packages/stg-evm-v2/src/utils
- https://github.com/stargate-protocol/stargate-v2/blob/main/packages/stg-evm-v2/src/StargateBase.sol
- https://github.com/stargate-protocol/stargate-v2/blob/main/packages/stg-evm-v2/src/StargateOFT.sol
- https://github.com/stargate-protocol/stargate-v2/blob/main/packages/stg-evm-v2/src/StargatePool.sol
- https://github.com/stargate-protocol/stargate-v2/blob/main/packages/stg-evm-v2/src/StargatePoolNative.sol

## Auditors:

- PopPunkOnChain
- 0xjustadev

## Findings (issues):

- https://github.com/stargate-protocol/stargate-v2/issues/115
- https://github.com/stargate-protocol/stargate-v2/issues/116
- https://github.com/stargate-protocol/stargate-v2/issues/117
- https://github.com/stargate-protocol/stargate-v2/issues/118
- https://github.com/stargate-protocol/stargate-v2/issues/124

- https://github.com/stargate-protocol/stargate-v2/issues/128

## Finding 1

File: StargateBase.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L63-L72

Optimize the `nonReentrantAndNotPaused` modifier for the non-revert case by checking if `status` is not `NOT_ENTERED` before checking the two revert cases for `PAUSED` and `ENTERED`. Skip caching the `status` storage variable on the stack since it is only read once for the non-revert path.

```
modifier nonReentrantAndNotPaused() {
    // On the first call to nonReentrant, _status will be _NOT_ENTERED
    if (status != NOT_ENTERED) {
        if (status == PAUSED) revert Stargate_Paused();
        if (status == ENTERED) revert Stargate_ReentrantCall();
    }
    // Any calls to nonReentrant after this point will fail
    status = ENTERED;
    _;
    status = NOT_ENTERED;
}
```

Saves 34 gas on every call that uses `nonReentrantAndNotPaused`.

## Finding 2

Files: StargateBase.sol, StargateOFT.sol, StargatePool.sol, StargatePoolNative.sol, LPToken.sol

Use `uint256` over `uint8` for decimals and check that `_tokenDecimals` in `StargateBase` is not greater than `uint8` max before type casting `_sharedDecimals` to `uint8` for storing in the immutable `sharedDecimals`. This avoids excess type checking for the `uint8` values to lower deployment costs.

```
constructor(address _token, uint256 _tokenDecimals, uint256 _sharedDecimals, address
    token = _token;
    if (_tokenDecimals > type(uint8).max || _tokenDecimals < _sharedDecimals) revert
    convertRate = 10 ** (_tokenDecimals - _sharedDecimals);
    sharedDecimals = uint8(_sharedDecimals);
```

```
        endpoint = ILayerZeroEndpointV2(_endpoint);
        localEid = endpoint.eid();
        _transferOwnership(_owner);
    }
```

Saves 135 gas on each deployment.

## Finding 3

File: StargateBase.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L187

Natspec specifies a revert if the destination chain is same as local but function does not implement a revert.

```
    /// @notice Set the baseFareMultiplierBps for a path.
    /// @dev Planner can set the fare, but the new fare will only be effective for the ne
    /// @dev It is possible for the Planner to set a high fare to grief the pathway. Dao
    /// @dev Reverts with InvalidPath if the destination chain is the same as local.
    /// @param _dstEid The destination chain LZ endpoint ID
    /// @param _baseFareMultiplierBps The new base fare multiplier to set
    function setBaseFareMultiplierBps(uint32 _dstEid, uint16 _baseFareMultiplierBps) exte
        paths[_dstEid].baseBusFareMultiplierBps = _baseFareMultiplierBps;
    }
```

## Finding 4

File: StargateBase.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L260-L261

Do not cache `hash` since it is only used once for comparing to the `unreceivedTokens` mapping value.

```
        bytes32 hash = ;
        if (unreceivedTokens[_guid][_index] !=
            keccak256(abi.encodePacked(_receiver, _amountLD, _composeIdx, _composeMsg))
        ) revert Stargate_UnreceivedTokenNotFound();
```

Saves ~3 gas on retry calls.

## Finding 5

File: StargateBase.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L303-L304

Do not cache `payload` since it is only used once to create the hash for `unreceivedTokens`.

```
unreceivedTokens[_guid][_index] = keccak256(abi.encodePacked(receiver, amount
```

Saves 4 gas on calls that get stored for retry.

## Finding 6

File: StargateBase.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L317-L318

Do not cache `unlimited` since it is only used once to check for invalid path.

```
if (paths[c.srcEid].decreaseCredit(c.amount)) revert Stargate_InvalidPath();
```

Saves ~3 gas on each iteration of the loop;

## Finding 7

File: StargateBase.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L5

Import and utilize Solady `SafeCastLib` for type casting instead of OpenZeppelin for more efficient type casting.

```
import { SafeCastLib } from "@solady/utils/SafeCastLib.sol";
...
    using SafeCastLib for uint256;
...
                oftFeeDetails[0] = OFTFeeDetail(-1 * _sd2ld(amountInSD - amountOutSD).to
...
                    oftFeeDetails[0] = OFTFeeDetail(_sd2ld(reward).toInt256(), "reward");
...
            amountSD = SafeCast.toUint64(_amountLD / convertRate);
```

Saves ~3 gas units per casting operation and reduces contract bytecode size by ~100 bytes.

## Finding 8

File: StargateBase.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L363 https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L372

If `quoteOFT` is intended to or may be used onchain, consider replacing the `"protocol fee"` and `"reward"` strings assigned to `OFTFeeDetail` structs with values from an enum or constants defined in a library. This will reduce the cost of executing this function, reduce the bytecode size of the contract and reduce the gas cost of interpreting the value in another contract.

## Savings varies depending on implementation and usage.

## Finding 9

File: StargateBase.sol

https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L222-L228 https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L458-L461

Return `isTaxi` as a named return value from `_inflowAndCharge` to avoid checking if the send is taxi mode twice.

```
...

        (uint64 amountInSD, uint64 amountOutSD, bool isTaxi, RideBusOptions memory rideBu
        oftReceipt = OFTReceipt(_sd2ld(amountInSD), _sd2ld(amountOutSD));

        // step 2: _send the token based on the mode
        MessagingFee memory messagingFee = _assertMessagingFee(_fee, oftReceipt.amountSer
        if (_isTaxiMode(_sendParam.oftCmd)) {

...

    function _inflowAndCharge(
        SendParam calldata _sendParam
    ) internal returns (uint64 amountInSD, uint64 amountOutSD, bool isTaxi, RideBusOptior
        isTaxi = _isTaxiMode(_sendParam.oftCmd);
```

Saves ~180 gas on `send` and `sendToken` calls.

## Finding 10

File: StargateBase.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateBase.sol#L533-L536

Cache `lzToken` on the stack to avoid a second SLOAD of the `lzToken` address.

```
    function _payLzToken(uint256 _lzTokenFee) internal {
        address _lzToken = lzToken;
        if (_lzToken == address(0)) revert Stargate_LzTokenUnavailable();
        Transfer.safeTransferTokenFrom(_lzToken, msg.sender, address(endpoint), _lzTokenF
    }
```

Saves ~100 gas units when paying in `lzToken` .

## Finding 1

File: StargateOFT.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargateOFT.sol#L29-L33

Do not cache the updated `_amount` value to remove dust, past the output directly into the `burnFrom` function since it is only used once.

```
function _inflow(address _from, uint256 _amount) internal virtual override returns (u
    amountSD = _ld2sd(_amount);
    IERC20Minter(token).burnFrom(_from, _sd2ld(amountSD)); // remove dust and burn
}
```

Saves ~23 gas on each inflow.

## Finding 1

File: StargatePool.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargatePool.sol#L124-L125

Create an empty `RideBusParams` object in memory instead of building it with an empty string and zero value.

```
    RideBusParams memory emptyRideBusParams;
    (uint64 amountOutSD, ) = _chargeFee(params, emptyRideBusParams, _ld2sd(_sendParam
```

Saves ~22 gas units on `redeemSend` calls.

## Finding 2

File: StargatePool.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargatePool.sol#L207-L223

Do not use named returns on the `_capReward` function so that the original values can be returned without moving them around on the stack.

```
function _capReward(
    uint64 _amountOutSD,
    uint64 _reward
) internal view returns (uint64 newAmountOutSD, uint64 newReward) {
```

```
        uint64 rewardCap = treasuryFee;
        if (_reward > rewardCap) {
            // exceeds cap, recalculate with new reward
            unchecked {
                return ((_amountOutSD - _reward + rewardCap), rewardCap);
            }
        }
        // lower than cap, return the original values
        return (_amountOutSD, _reward);
    }
```

Saves 5 gas when reward is capped, saves 14 gas when reward is not capped.

## Finding 3

File: StargatePool.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargatePool.sol#L235-L239

Do not cache the updated `_amountLD` value to remove dust, past the output directly into the `safeTransferTokenFrom` function since it is only used once.

```
    function _inflow(address _from, uint256 _amountLD) internal virtual override returns
        amountSD = _ld2sd(_amountLD);
        Transfer.safeTransferTokenFrom(token, _from, address(this), _sd2ld(amountSD)); //
    }
```

Saves ~23 gas on each inflow.

## Finding 1

File: StargatePoolNative.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/StargatePoolNative.sol#L73-L77

Adjust the logic of the `_assertMsgValue` function to only apply the not a single time.

```
    function _assertMsgValue(uint256 _amountLD) internal view override {
        // msg.value should be exactly the same as the amountLD and not have dust
```

```
        // _sd2ld(_ld2sd(_amountLD))) removes the dust if any
        if (!(_amountLD == msg.value && _amountLD == _sd2ld(_ld2sd(_amountLD)))) revert S
    }
```

Saves 6 gas when `_amountLD` matches `msg.value` for both revert and success cases. Saves 3 gas on revert case when `_amountLD` does not match `msg.value`.

## Finding 1

File: CreditMessaging.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/messaging/CreditMessaging.sol#L22-L44

Track the number of succeeded batches for indexing instead of subtracting `failedBatchNum` from `i`.

```
        uint256 succeededBatchNum = 0;
        uint256 failedBatchNum = 0;
        uint128 totalCreditNum = 0; // total number of credits in all batches
        for (uint256 i = 0; i < _creditBatches.length; i++) {
            CreditBatch calldata batch = _creditBatches[i];
            address stargate = _safeGetStargateImpl(batch.assetId);
            try ICreditMessagingHandler(stargate).sendCredits(_dstEid, batch.credits) {
                succeededBatches[succeededBatchNum] = batch;
                totalCreditNum += uint128(batch.credits.length); // safe cast
                unchecked {
                    ++succeededBatchNum;
                }
            } catch {
                failedBatches[failedBatchNum] = batch;
                unchecked {
                    failedBatchNum++;
                }
            }
        }

        // update the length of arrays and emit event if there are failed batches
        if (failedBatchNum > 0) {
            assembly {
                mstore(succeededBatches, succeededBatchNum)
                mstore(failedBatches, failedBatchNum)
            }
            emit CreditsFailedToSend(_dstEid, failedBatches);
        }
```

Gas savings vary based on number of batches, failures, and credit updates in each batch. With four batches of two updates each and no failures gas savings is ~220 gas units, adding one batch that fails is ~290 gas savings, 20 batches with 20% fails is ~1170 gas units saved.

## Finding 2

File: CreditMessaging.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/messaging/CreditMessaging.sol#L26-L27 https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/messaging/CreditMessaging.sol#L75-L76

Do not cache the `stargate` address, use it directly in the interface casting to avoid duplicating the item on the stack.

```
            try ICreditMessagingHandler(_safeGetStargateImpl(batch.assetId)).sendCredits(
...
            ICreditMessagingHandler(_safeGetStargateImpl(creditBatches[i].assetId))
```

Saves ~3 gas on each call.

## Finding 3

File: CreditMessaging.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/messaging/CreditMessaging.sol#L73-L77

Cache the length of the `creditBatches` memory array for loop iteration and cache the memory pointer for each item index while iterating over the loop;

```
        CreditBatch[] memory creditBatches = CreditMsgCodec.decode(_message);
        uint256 creditBatchesLength = creditBatches.length;
        for (uint256 i = 0; i < creditBatchesLength; i++) {
            CreditBatch memory creditBatch = creditBatches[i];
            address stargate = _safeGetStargateImpl(creditBatch.assetId);
            ICreditMessagingHandler(stargate).receiveCredits(_origin.srcEid, creditBatch.
        }
```

Gas savings will vary based on the number of batches in the call.

## Finding 1

File: TokenMessaging.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/messaging/TokenMessaging.sol#L188-L189

Cache the `_payloads.length` value on the stack to avoid extra MLOADs for each loop iteration.

```
uint256 _payloadsLength = payloads.length;
for (uint256 i = 0; i < _payloadsLength; i++) {
```

## Savings varies depending on length of array.

## Finding 2

File: TokenMessaging.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/messaging/TokenMessaging.sol#L196-L197

Do not cache `success`, use the return value directly in the if statement to determine if `NativeDropFailed` should be emitted.

```
if (!Transfer.transferNative(receiver, _nativeDropAmount, true)) emit Nat
```

Saves 3 gas per native drop.

## Finding 3

File: TokenMessagingOptions.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/messaging/TokenMessagingOptions.sol#L83-L97

Cache the `_payloads.length` value on the stack to avoid extra MLOADs for each loop iteration.

Cache the `_payloads[i]` memory pointer to avoid calculating the memory pointer location for each loop iteration.

```
uint256 _payloadsLength = payloads.length;
for (uint256 i = 0; i < _payloadsLength; i++) {
    TransferPayloadDetails memory _payload = _payloads[i];
    // calculate total gas for lzReceive
    totalGas += _safeGetGasLimit(_eid, _payload.assetId);

    // append lzCompose option
    if (_payload.composeMsg.length > 0) {
        buf.appendUint8(EXECUTOR_WORKER_ID);
        bool hasLzComposeValue = _payload.lzComposeValue > 0;
        buf.appendInt(hasLzComposeValue ? OPTION_LZCOMPOSE_PARAMS_SIZE : OPTION_L
        buf.appendUint8(OPTION_TYPE_LZCOMPOSE);
        buf.appendInt(composeIdx++, 2);
        buf.appendInt(_payload.lzComposeGas, 16);
        if (hasLzComposeValue) buf.appendInt(_payload.lzComposeValue, 16);
    }
}
```

Gas savings will vary based on the number of payloads in the array.

## Finding 1

File: Bus.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/libs/Bus.sol#L87

Use unchecked math for calculating the refund amount since it is already checked with `if (fare < msg.value)`.

```
unchecked {
    refund = msg.value - fare;
}
```

## Finding 2

File: Bus.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/libs/Bus.sol#L14-L15 https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/libs/Bus.sol#L104-L110 https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/libs/Bus.sol#L159-L160

Use a storage mapping over an array for storing the bus passenger hash chain. This allows for getting an uninitialized zero bytes slot for the first ticket and functions effectively as an array without adding the `ticketId == 0` and `if (ticketId < busCapacity)` overhead to every function call.

```
    mapping(uint256 => bytes32) hashChain; // hash chain of the passengers
...
        unchecked {
            bytes32 lastHash = _bus.hashChain[(ticketId - 1) % _busCapacity];
            bytes32 newHash = keccak256(abi.encodePacked(lastHash, passengerBytes));
            _bus.hashChain[ticketId % _busCapacity] = newHash;
        }
...
        bytes32 lastHash;
        unchecked {
            lastHash = _bus.hashChain[(startTicketId - 1) % _busCapacity];
        }
```

Saves ~5520 gas on every ride call up to the first full bus (~620 gas each additional ticket for the first full bus created within the same transaction), saves ~415 gas on every ride call after the first bus is full.

## Finding 1

File: CreditMsgCodec.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/libs/CreditMsgCodec.sol#L41 https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/libs/CreditMsgCodec.sol#L45-L46

Cache the length of the `_creditBatches` and `batch.credits` memory arrays.

```
        uint256 _creditBatchesLength = _creditBatches.length;
        for (uint256 i = 0; i < _creditBatchesLength; i++) {
```

```
...
            uint256 _creditBatchCreditsLength = batch.credits.length;
            for (uint256 j = 0; j < _creditBatchCreditsLength; j++) {
```

## Savings varies depending on length of arrays.

## Finding 2

File: Path.sol https://github.com/LayerZero-Labs/stargate-v2/blob/322d74b56d39a421cee6406940968eb7dd2e245b/packages/stg-evm-v2/src/libs/Path.sol#L53-L57

The `tryDecreaseCredit` function can return an unexpected value if the PathLib is used incorrectly by returning a decrease amount when the current credit value is set to `UNLIMITED_CREDIT`. Calling `decreaseCredit` after performing an SLOAD on the current credit value and performing checks on the decrease amount compared to current credit duplicates the SLOAD and amount check.

Instead of calling `decreaseCredit`, perform the logic of decreasing the credit amount inside of `tryDecreaseCredit`.

```
    function tryDecreaseCredit(Path storage _path, uint64 _amountSD) internal returns (ui
        uint64 currentCredit = _path.credit;
        if (currentCredit == UNLIMITED_CREDIT) return 0;
        if (_amountSD > currentCredit) {
            _path.credit = 0;
            return currentCredit;
        }
        unchecked {
            _path.credit = currentCredit - _amountSD;
        }
        return _amountSD;
    }
```

Saves 213 gas when the amount is less than current credit, saves 209 gas when amount is greater than the current credit.

## Remediations:

https://github.com/stargate-protocol/stargate-v2/pull/161